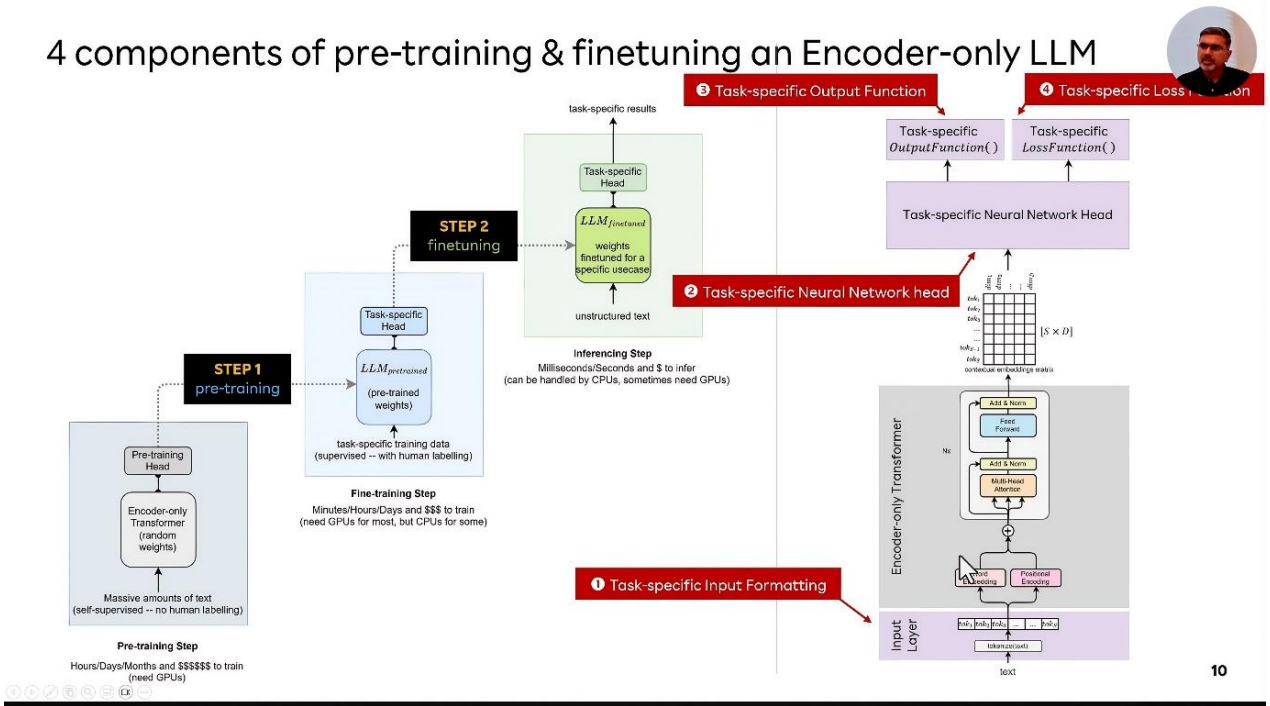
PRETRAINING OF ENCODER ONLY TRANSFORMER



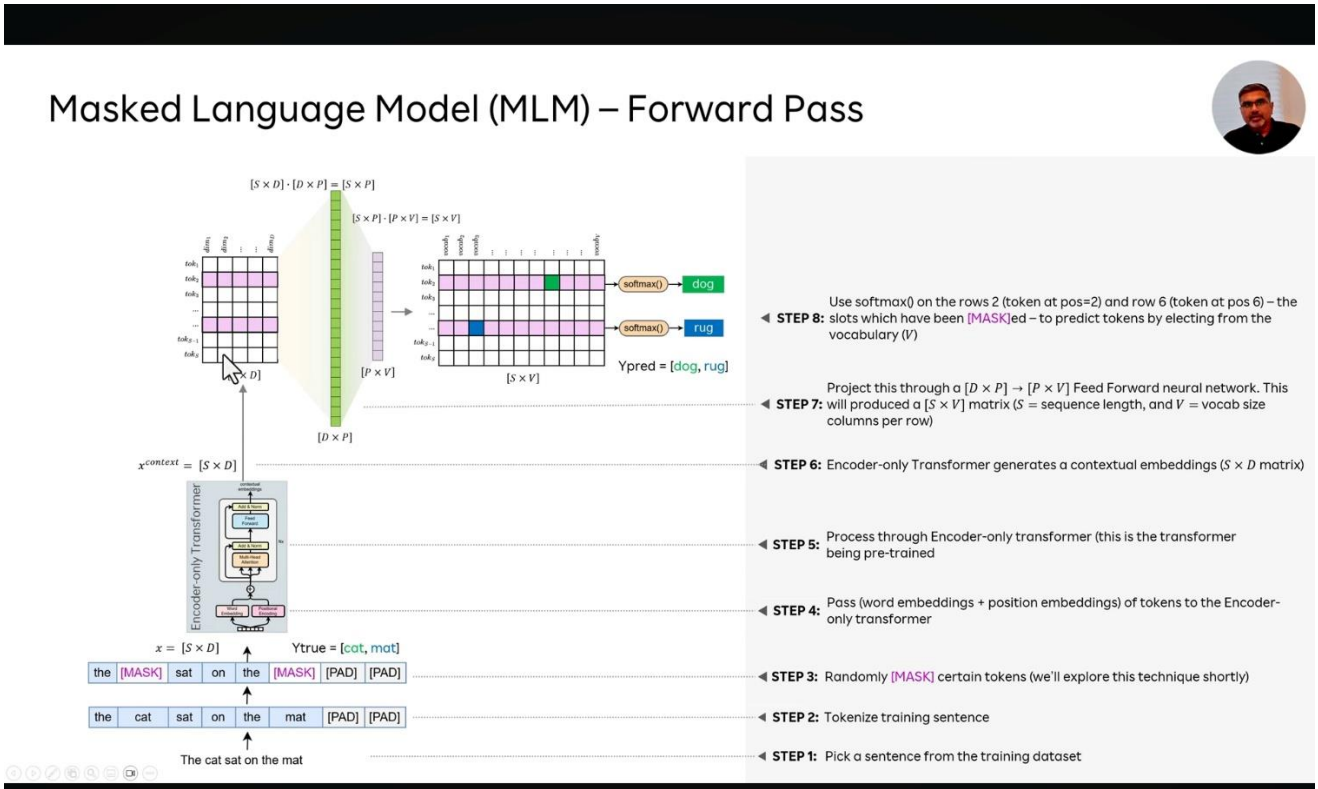
Various pretraining techniques are –

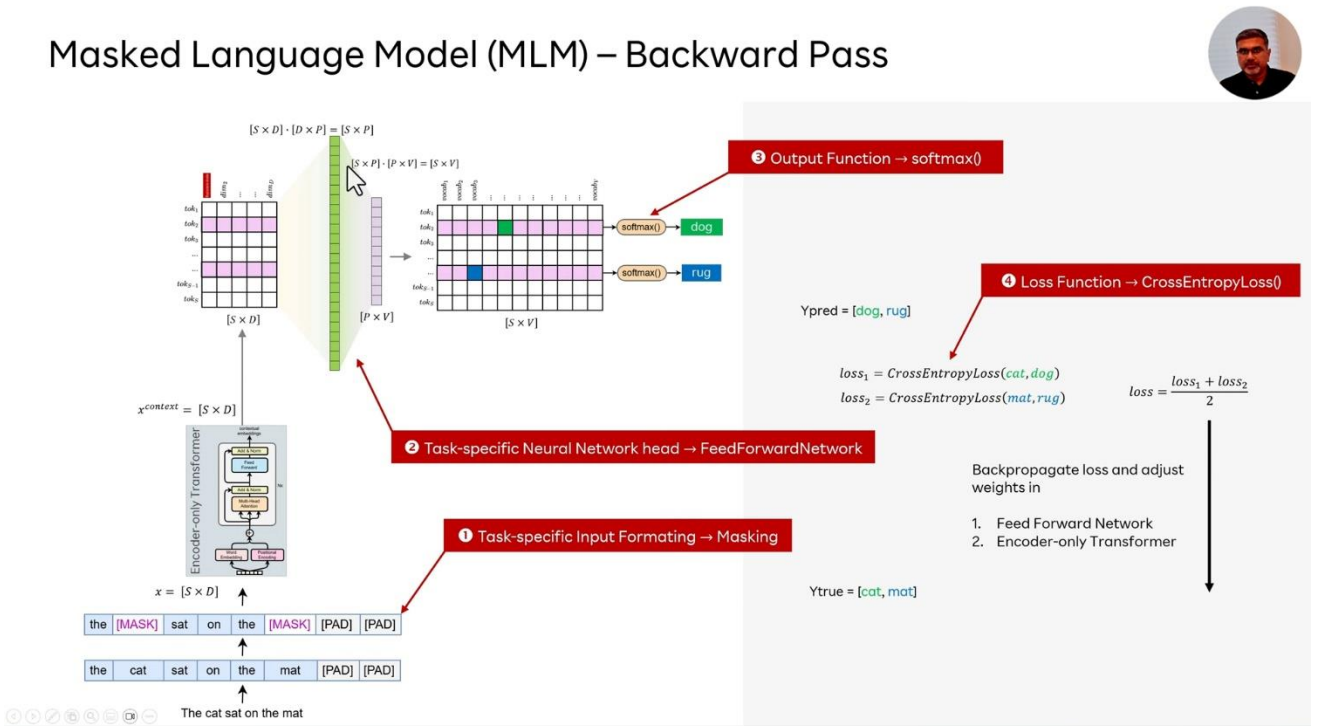
1. MLM (Masked Language Model)
2. RTD (Replaced Token Detection)
3. NSP (Next Sentence Prediction)
4. SOP (Sentence Order Prediction)

MLM (Masked Language Model)

Masked Language Modeling (MLM) is a self-supervised learning technique where some tokens in a sentence are randomly masked, and the model is trained to predict the original tokens using the surrounding context.

Working steps -





How to determine which tokens to [MASK] ?

Step 1: Random Selection

Select ~15% of tokens uniformly at random from the input sequence.

Step 2: Apply Masking Strategy

For each selected token:

80% → Replace with [MASK]

10% → Replace with random token

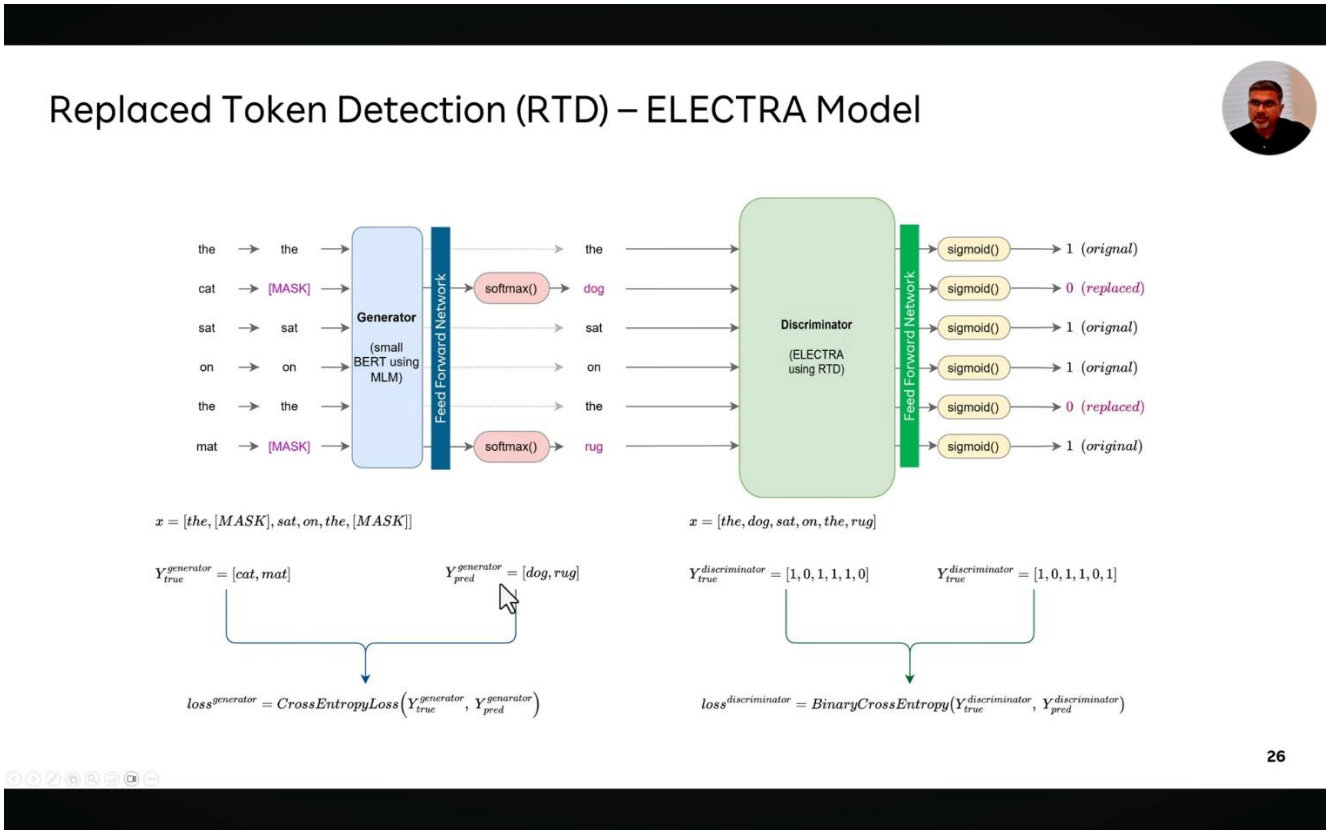
10% → Keep unchanged

RTD (Replaced Token Detection)

RTD trains a model to detect replaced tokens in a sequence, enabling efficient and fully supervised learning at every token position.

RTD (Replaced Token Detection) – Working Steps

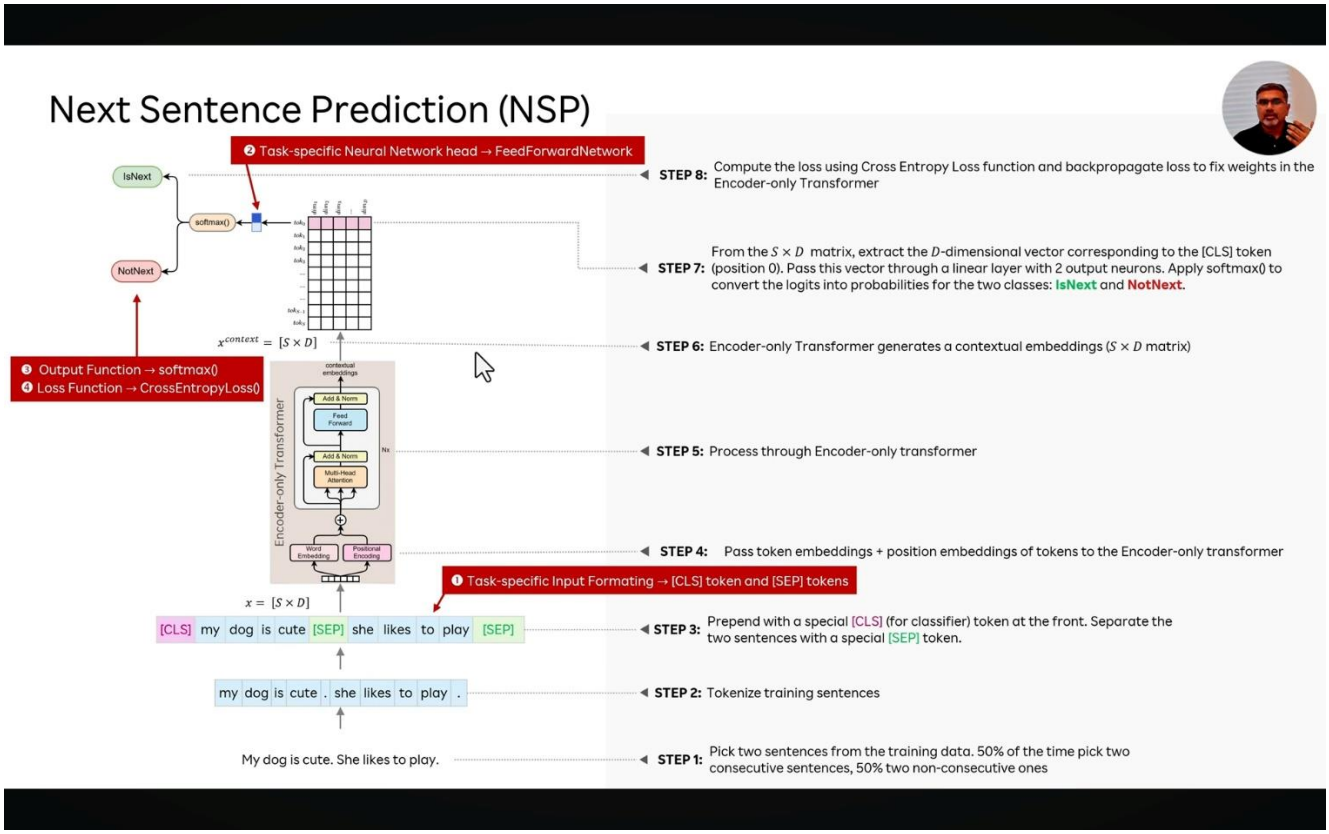
1. **Input Sentence** - A raw sentence is taken as input for processing.
2. **Tokenization** - The sentence is tokenized into subword tokens with special tokens like [CLS] and [SEP].
3. **Masking Tokens** - Some tokens are randomly selected and replaced with [MASK] tokens.
4. **Generator Prediction** - A small generator model predicts possible tokens for the masked positions.
5. **Token Replacement** - Masked tokens are replaced with generator-predicted tokens (correct or incorrect).
6. **Corrupted Sequence** - A new sequence is formed containing both original and replaced tokens.
7. **Discriminator Input** - The corrupted sequence is fed into the discriminator model.
8. **Token-Level Classification** - The discriminator predicts for each token whether it is original or replaced.
9. **Loss Computation** - Binary classification loss is computed for each token (real vs fake).
10. **Backpropagation** - The discriminator (and sometimes generator) weights are updated to improve detection.



NSP (Next Sentence Prediction)

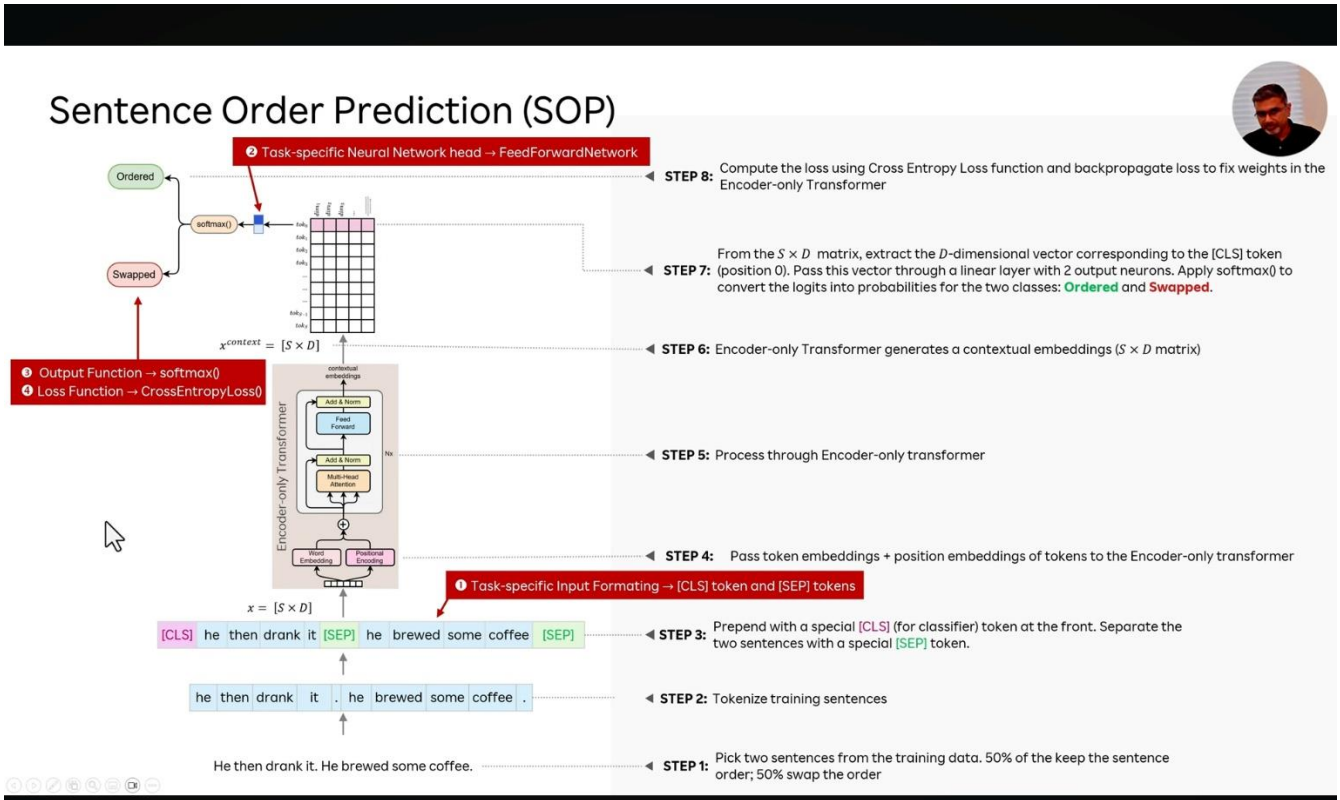
NSP trains a model to determine whether two sentences are sequentially related or randomly paired.

Working –



SOP (Sentence Order Prediction)

SOP is a binary task that trains a model to detect whether two consecutive text segments are in the correct order or swapped, improving discourse-level understanding.



COMPARISON OF VARIOUS ENCODER ONLY TRANSFORMER MODELS

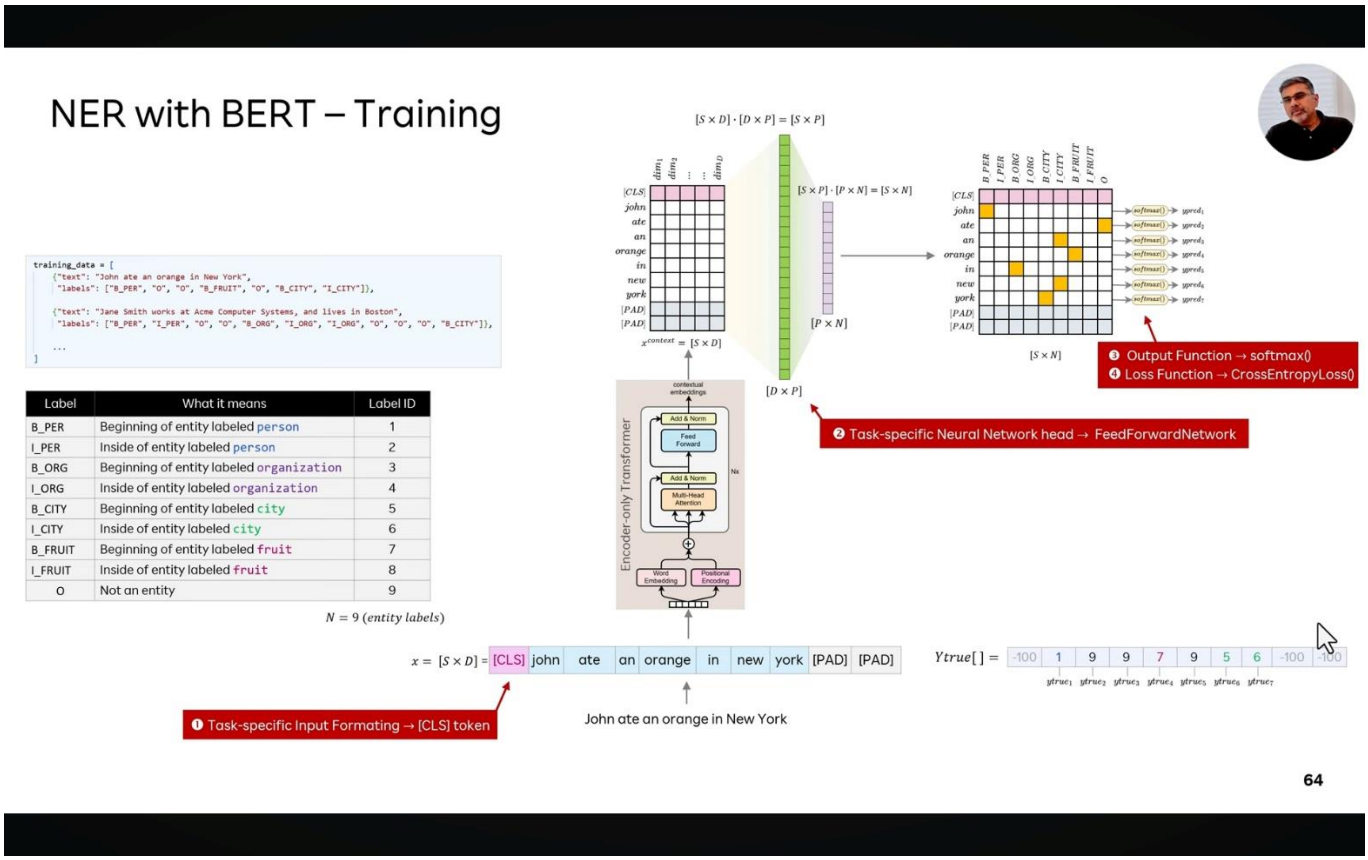
Comparison of different pre-trained BERT models

Model Name	MLM	RTD	NSP	SOP	Model Size (Parameters)	Training Dataset Type & Size	Vocab Size	Year Introduced, Institution	HuggingFace Model
BERT	✓	×	✓	×	Base: 110M Large: 340M	BookCorpus (800M words) + English Wikipedia (2.5B words) (Total ~3.3 Billion words)	30,522	2018 Google	`bert-base-uncased`
RoBERTa	✓	×	×	×	Base: 125M Large: 355M	Significantly larger dataset (160 GB of text) including: BookCorpus, English Wikipedia, CC-News, OpenWebText, Stories	50,265	2019 Facebook	`roberta-base`
ALBERT	✓	×	×	✓	Base: 12M Large: 18M XXLarge: 235M	BookCorpus + English Wikipedia (similar to BERT)	30,000	2019 Google	`albert-base-v2`
ELECTRA	✓	✓	×	×	Small: 14M Base: 110M Large: 335M	BookCorpus + English Wikipedia / OpenWebText (similar scale to BERT, but highly efficient training)	30,522	2020 Google	`google/electra-base-discriminator`
DistilBERT	✓	×	×	×	Base: 66M (60% smaller than BERT-base)	Distilled from BERT on BERT's original corpus (BookCorpus + English Wikipedia)	30,522	2019 Hugging Face	`distilbert-base-uncased`
DeBERTa	✓	×	×	×	Base: 140M Large: 390M v3 Large: 278M	Large-scale text corpus (~160GB of text) including Wikipedia, BookCorpus, OpenWebText, CC-News. Some variants trained on ~80GB or ~400GB for specific languages.	50,265	2020 (v3 in 2021) Microsoft	`microsoft/deberta-base`
TinyBERT	✓	×	×	×	4M-15M Very small parameter counts via distillation	Distilled from BERT, utilizing BERT's original training corpus. Not pre-trained from scratch on a new corpus.	30,522	2020 Huawei	`huawei-noah/TinyBERT_General_4L_312D`

NER

NER is a sequence labeling task where each token in a sentence is classified into entity categories like person, location, or organization.

It uses BIO (Beginning Inside Outside) tags for the identification of the entities.



CRF (Conditional Random Field) is a probabilistic model used for sequence labeling tasks in Natural Language Processing.

Why to add CRF on top of BERT for NER?

Because CRF captures dependencies between labels and ensures valid sequences.

How would you improve NER performance?

1. Better preprocessing
2. Domain-specific fine-tuning
3. Data augmentation
4. Hyperparameter tuning
5. Error analysis

SIAMESE NETWORK VS BI-ENCODER VS SENTENCE TRANSFORMER

1. Siamese Network (Training Concept)

A Siamese network is a training framework where two or more identical models with shared weights process different inputs, and the model is trained using a similarity-based objective (e.g., contrastive or triplet loss) to learn an embedding space.

What this implies:

Focus = how the model is trained

Goal = learn similarity / distance

Not tied to NLP only (used in vision, speech, etc.)

2. Bi-Encoder (Architecture / Inference Concept)

A bi-encoder is a model architecture where two inputs are encoded independently using the same encoder into fixed embeddings, and their relationship is computed using a similarity function (e.g., cosine similarity), enabling efficient large-scale comparison.

What this implies:

Focus = how the model is used

Goal = fast retrieval / matching

Typically used at inference time

3. Sentence Transformer (Specific Model)

A Sentence Transformer (e.g., Sentence-BERT) is a bi-encoder model built on transformers like BERT that is trained using a Siamese or triplet setup to produce semantically meaningful sentence embeddings optimized for similarity tasks.

What this implies:

Combines both Siamese training and Bi-encoder architecture

Goal = high-quality embeddings out-of-the-box

Objectives

- Siamese Network → To learn an embedding space where similar inputs are close and dissimilar ones are far apart, enabling similarity-based applications like verification, matching, and duplicate detection.
- Bi-Encoder → To independently encode inputs into comparable embeddings for scalable similarity computation, enabling efficient large-scale retrieval and recommendation systems.
- Sentence Transformer (SBERT) → To generate semantically meaningful sentence embeddings optimized for similarity learning, enabling accurate semantic search, clustering, and RAG applications.

Why Sentence Transformers Exist ?

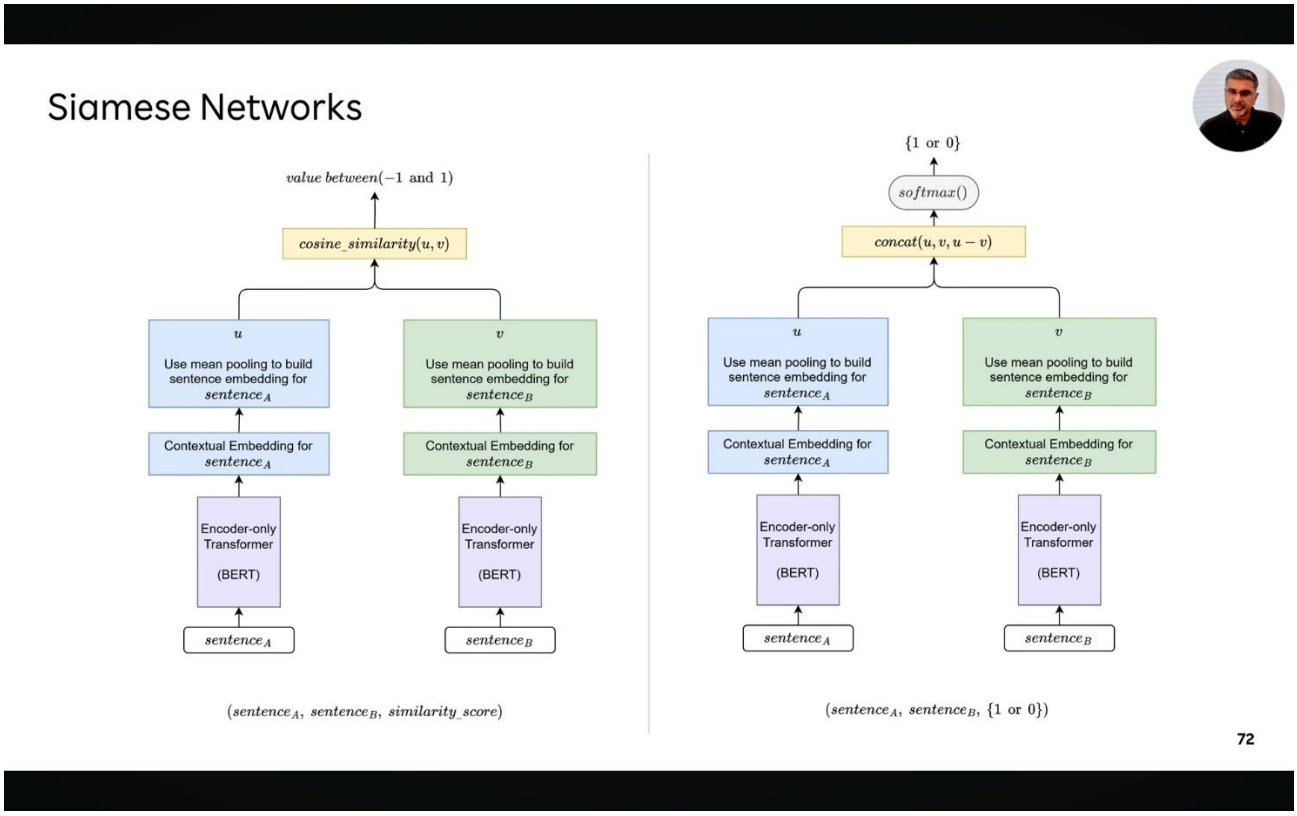
Vanilla BERT is not suitable for sentence similarity tasks directly because:

- It produces token-level embeddings, not sentence-level
- Using [CLS] embedding without fine-tuning is weak for semantic similarity
- Pairwise comparison with BERT is computationally expensive ($O(N^2)$)

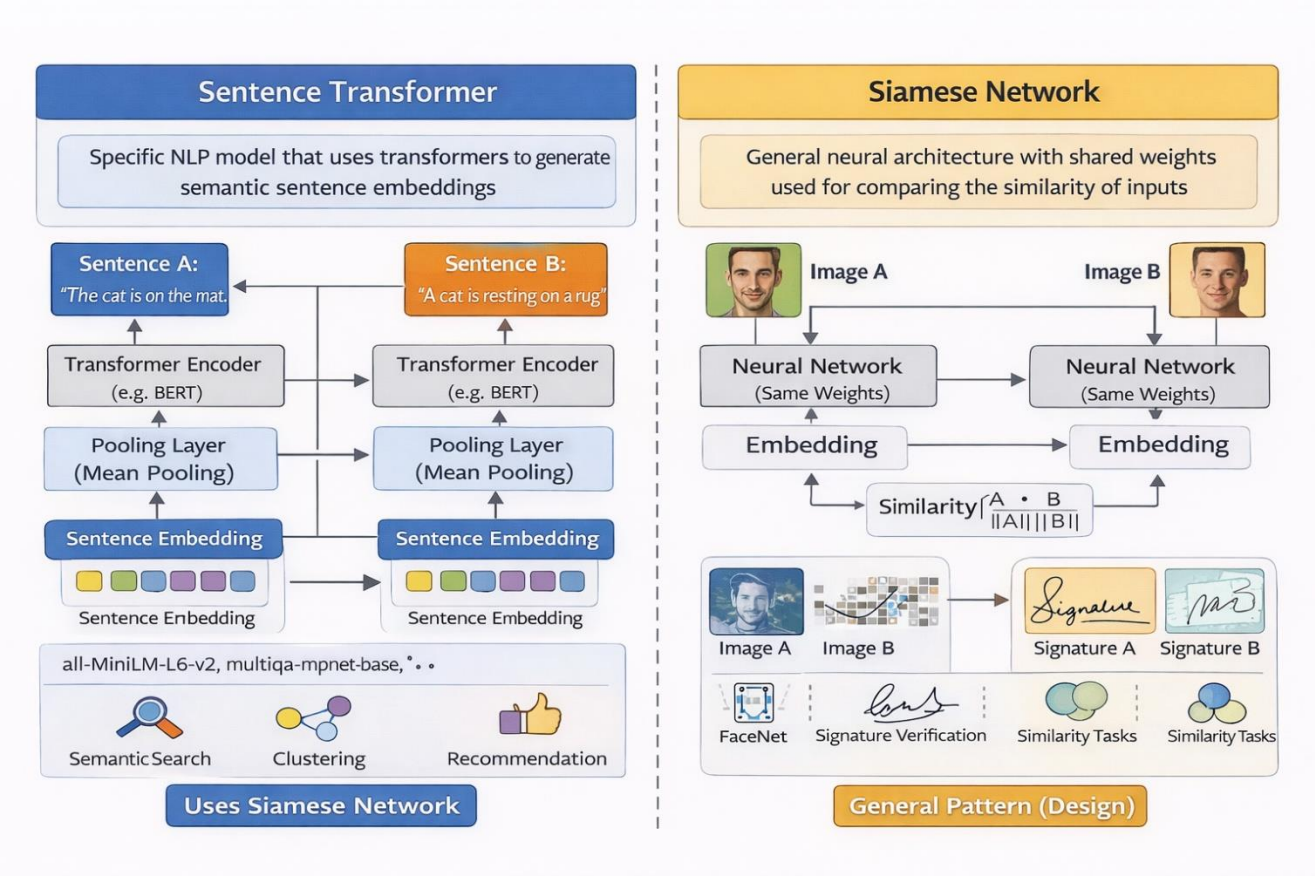
Sentence Transformers solve this by:

- Producing meaningful sentence embeddings
- Enabling efficient similarity search ($O(N)$) using vector indexing)

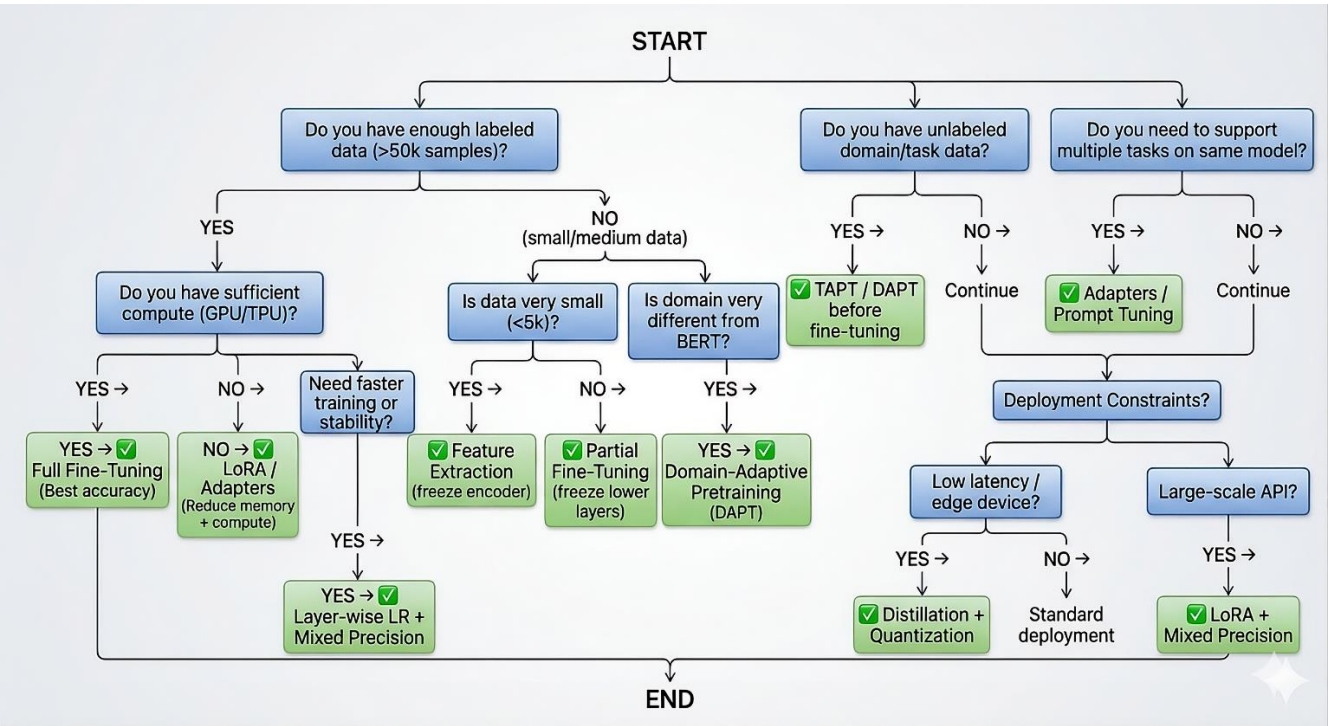
Working steps are –



Above image is actually a sentence transformer (SBERT), not a Siamese network



FINE TUNING OF ENCODER ONLY TRANSFORMERS



In above image, Continue = skip this branch, move forward in decision flow”

I choose fine-tuning strategy based on data size, compute, domain shift, and deployment constraints. For large data I use full fine-tuning, for limited compute I use LoRA or adapters, and for domain-specific tasks I apply DAPT before fine-tuning. For deployment, I optimize using distillation and quantization.”